```
#!/usr/bin/env python3
"""Gold Watch - alertes news + recaps XAUUSD via Telegram.

Modes: test | alert | morning | preview | weekly
"""
import os
import sys
import json
import re
import hashlib
import calendar
import datetime as dt
from zoneinfo import ZoneInfo

import requests
import feedparser

TZ = ZoneInfo(os.getenv("TZ_NAME", "America/Toronto"))
MODEL = os.getenv("CLAUDE_MODEL", "claude-haiku-5-5")
IMPACT_MIN = int(os.getenv("IMPACT_MIN", "2"))  # 3 = seulement les news intenses
SEEN_FILE = "seen.json"
UA = {"User-Agent": "Mozilla/5.0 (GoldWatch)"}

FEEDS = {
    "Gold": "https://news.google.com/rss/search?q=gold+price+OR+XAUUSD+OR+%22gold
    "Fed/USD": "https://news.google.com/rss/search?q=Fed+OR+inflation+OR+%22US+do
    "Macro/Geo": "https://news.google.com/rss/search?q=%22central+bank%22+gold+OR
    "FXStreet": "https://www.fxstreet.com/rss/news",
    "Investing": "https://www.investing.com/rss/news_11.rss",
}

HIGH_KW = ["fomc", "powell", "fed ", "cpi", "nfp", "payroll", "rate cut", "rate h
           "war", "attack", "missile", "sanction", "tariff", "central bank", "gol
           "gold surges", "gold plunges", "emergency", "default"]
MID_KW = ["gold", "xau", "inflation", "dollar", "yields", "treasury", "pce", "gdp
          "jobless", "ecb", "boj", "pboc", "geopolit", "safe haven"]


# ---------- utilitaires ----------
def send_telegram(text):
    token, chat = os.environ["TELEGRAM_TOKEN"], os.environ["TELEGRAM_CHAT_ID"]
    for i in range(0, len(text), 3900):
        r = requests.post(
```

```
            f"https://api.telegram.org/bot{token}/sendMessage",
            json={"chat_id": chat, "text": text[i:i + 3900], "disable_web_page_pr
            timeout=30,
        )
        r.raise_for_status()


def claude(prompt, max_tokens=1800):
    key = os.getenv("ANTHROPIC_API_KEY")
    if not key:
        return None
    r = requests.post(
        "https://api.anthropic.com/v1/messages",
        headers={"x-api-key": key, "anthropic-version": "2023-06-01", "content-ty
        json={"model": MODEL, "max_tokens": max_tokens, "messages": [{"role": "us
        timeout=120,
    )
    r.raise_for_status()
    return r.json()["content"][0]["text"]


def load_seen():
    try:
        with open(SEEN_FILE) as f:
            return json.load(f)
    except Exception:
        return []


def save_seen(seen):
    with open(SEEN_FILE, "w") as f:
        json.dump(seen[-1500:], f)


def uid(s):
    return hashlib.md5(s.lower().strip().encode()).hexdigest()[:16]


# ---------- donnees ----------
def fetch_news(hours):
    cutoff = dt.datetime.now(dt.timezone.utc) - dt.timedelta(hours=hours)
    items, titles = [], set()
    for source, url in FEEDS.items():
        try:
            feed = feedparser.parse(url, request_headers=UA)
        except Exception as e:
```

```
            print("feed error", source, e)
            continue
        for e in feed.entries:
            ts = e.get("published_parsed")
            if not ts:
                continue
            t = dt.datetime.fromtimestamp(calendar.timegm(ts), dt.timezone.utc)
            title = e.get("title", "").strip()
            if t < cutoff or not title or title.lower() in titles:
                continue
            titles.add(title.lower())
            items.append({"title": title, "link": e.get("link", ""), "source": so
    items.sort(key=lambda x: x["ts"], reverse=True)
    return items


def calendar_events():
    raw = []
    for f in ("thisweek", "nextweek"):
        try:
            r = requests.get(f"https://nfs.faireconomy.media/ff_calendar_{f}.json
            r.raise_for_status()
            raw += r.json()
        except Exception as e:
            print("calendar error", f, e)
    out, keys = [], set()
    for e in raw:
        try:
            t = dt.datetime.fromisoformat(e["date"]).astimezone(TZ)
        except Exception:
            continue
        key = (e.get("title"), e.get("date"), e.get("country"))
        if key in keys:
            continue
        keys.add(key)
        impact, country = e.get("impact"), e.get("country")
        if impact == "High" or (impact == "Medium" and country == "USD"):
            out.append({"t": t, "country": country, "title": e.get("title"), "imp
                        "forecast": e.get("forecast", ""), "previous": e.get("pre
    out.sort(key=lambda x: x["t"])
    return out


def fmt_event(e, with_day=False):
    day = e["t"].strftime("%a %d/%m ") if with_day else ""
    star = "🔴" if e["impact"] == "High" else "🟠"
```

```
    return f"{star} {day}{e['t'].strftime('%H:%M')} {e['country']} {e['title']} (


def snapshot():
    lines = []
    for name, sym in (("Gold futures", "GC=F"), ("DXY", "DX-Y.NYB"), ("US 10 ans"
        try:
            r = requests.get(f"https://query1.finance.yahoo.com/v8/finance/chart/
                             headers=UA, timeout=20)
            res = r.json()["chart"]["result"][0]
            closes = [c for c in res["indicators"]["quote"][0]["close"] if c is n
            last, prev, first = closes[-1], closes[-2], closes[0]
            lines.append(f"{name}: {last:.2f} ({(last / prev - 1) * 100:+.2f}% jo
        except Exception as e:
            print("snapshot error", sym, e)
    return "\n".join(lines) or "Données de marché indisponibles"


# ---------- scoring ----------
def keyword_score(title):
    t = title.lower()

    def has(keywords):
        # début de mot seulement (évite "war" dans "award", garde "tariffs")
        return any(re.search(r"\b" + re.escape(k.strip()), t) for k in keywords)

    if has(HIGH_KW):
        return {"impact": 3, "bias": "?", "why": ""}
    if has(MID_KW):
        return {"impact": 2, "bias": "?", "why": ""}
    return {"impact": 1, "bias": "?", "why": ""}


def score_news(items):
    """Retourne la liste d'items enrichis de impact (1-3), bias, why."""
    items = items[:40]
    result = None
    if os.getenv("ANTHROPIC_API_KEY") and items:
        numbered = "\n".join(f"{i}. {it['title']}" for i, it in enumerate(items))
        prompt = (
            "Tu es analyste macro spécialisé dans l'or (XAUUSD). Pour chaque titr
            "potentiel sur le prix de l'or: 3 = intense (Fed, CPI/NFP, choc géopo
            "violent), 2 = modéré, 1 = faible ou hors sujet. Donne aussi le biais
            "neutre) et une raison de 12 mots max en français.\n"
            'Réponds UNIQUEMENT avec un tableau JSON: [{"i":0,"impact":3,"bias":"
            + numbered
```

```
        )
        try:
            txt = claude(prompt, 3000)
            data = json.loads(txt[txt.index("["): txt.rindex("]") + 1])
            result = {d["i"]: d for d in data}
        except Exception as e:
            print("scoring error, fallback mots-clés:", e)
    for i, it in enumerate(items):
        s = (result or {}).get(i) or keyword_score(it["title"])
        it["impact"] = int(s.get("impact", 1))
        it["bias"] = s.get("bias", "?")
        it["why"] = s.get("why", "")
    return items


# ---------- modes ----------
def mode_alert():
    seen = load_seen()
    seen_set = set(seen)
    new = [it for it in fetch_news(3) if uid(it["title"]) not in seen_set]
    if new:
        scored = score_news(new)
        for it in new:
            seen.append(uid(it["title"]))
        keep = sorted([it for it in scored if it["impact"] >= IMPACT_MIN], key=la
        if keep:
            icon = {3: "🔴", 2: "🟠", 1: "⚪"}
            bias = {"bullish": "📈", "bearish": "📉"}
            msg = ["🥇 GOLD ALERT"]
            for it in keep[:8]:
                msg.append(f"\n{icon[it['impact']]} {bias.get(it['bias'], '➖')}
                           + (f"\n   → {it['why']}" if it["why"] else "") + f"\n
            send_telegram("\n".join(msg))
    # avertissement avant les gros événements du calendrier
    now = dt.datetime.now(TZ)
    for e in calendar_events():
        mins = (e["t"] - now).total_seconds() / 60
        key = "ev:" + uid(f"{e['title']}{e['t'].isoformat()}{e['country']}")
        if 0 < mins <= 35 and key not in seen_set:
            send_telegram(f"⏰ Dans {int(mins)} min\n{fmt_event(e)}")
            seen.append(key)
    save_seen(seen)


def mode_morning():
    now = dt.datetime.now(TZ)
```

```
    news = score_news(fetch_news(16))
    top = [n for n in news if n["impact"] >= 2][:25]
    today = [e for e in calendar_events() if e["t"].date() == now.date()]
    prompt = (
        f"Nous sommes le {now.strftime('%A %d %B %Y')}. Écris en français, en tex
        "un recap du matin pour un trader d'or (XAUUSD) qui cherche surtout des s
        "1) BIAIS FONDAMENTAL DU JOUR (buy / sell / neutre, avec 2-3 raisons)\n"
        "2) POINTS CRUCIAUX (3 à 5 puces max: ce qui peut faire bouger l'or aujou
        "3) ÉVÉNEMENTS DU JOUR et comment les trader (avant/après la publication)
        "4) RISQUES contre le biais d'achat\n"
        "Reste concret, 250 mots max. N'invente aucun prix ni niveau: utilise seu
        "Termine par: Pas un conseil financier.\n\n"
        f"MARCHÉ:\n{snapshot()}\n\nCALENDRIER AUJOURD'HUI:\n"
        + ("\n".join(fmt_event(e) for e in today) or "Rien d'important")
        + "\n\nNEWS (impact/biais):\n"
        + "\n".join(f"[{n['impact']}|{n['bias']}] {n['title']}" for n in top)
    )
    txt = claude(prompt)
    if not txt:
        txt = ("Clé ANTHROPIC_API_KEY absente, voici les données brutes.\n\n" + s
               + "\n".join(fmt_event(e) for e in today) + "\n\n" + "\n".join(n["t
    send_telegram("🌄 RECAP GOLD DU MATIN\n\n" + txt)


def mode_preview():
    now = dt.datetime.now(TZ)
    evs = [e for e in calendar_events() if now <= e["t"] <= now + dt.timedelta(da
    news = score_news(fetch_news(24))
    prompt = (
        "Écris en français, en texte brut, l'aperçu de la semaine à venir pour l'
        "1) THÈME DE LA SEMAINE, 2) ÉVÉNEMENTS CLÉS jour par jour (les plus impor
        "3) SCÉNARIO HAUSSIER vs BAISSIER (fondamental), 4) MOMENTS PIÈGES (volat
        "300 mots max, aucun prix inventé. Termine par: Pas un conseil financier.
        f"MARCHÉ:\n{snapshot()}\n\nCALENDRIER:\n" + "\n".join(fmt_event(e, True)
        + "\n\nNEWS RÉCENTES:\n" + "\n".join(f"[{n['impact']}|{n['bias']}] {n['ti
    )
    txt = claude(prompt) or ("Calendrier de la semaine:\n" + "\n".join(fmt_event(
    send_telegram("📆 APERÇU DE LA SEMAINE - GOLD\n\n" + txt)


def mode_weekly():
    now = dt.datetime.now(TZ)
    evs = [e for e in calendar_events() if now - dt.timedelta(days=5) <= e["t"] <
    news = score_news(fetch_news(96))
    top = sorted(news, key=lambda x: -x["impact"])[:25]
    prompt = (
```

```
        "Écris en français, en texte brut, le recap de fin de semaine pour l'or (
        "1) CE QUI A MENÉ LA SEMAINE (3 moteurs max), 2) ÉVÉNEMENTS MARQUANTS et
        "3) CE QUI RESTE EN SUSPENS pour la semaine prochaine, 4) BIAIS FONDAMENT
        "300 mots max, aucun prix inventé. Termine par: Pas un conseil financier.
        f"MARCHÉ (5 jours):\n{snapshot()}\n\nÉVÉNEMENTS MAJEURS:\n" + "\n".join(f
        + "\n\nNEWS DE LA SEMAINE:\n" + "\n".join(f"[{n['impact']}|{n['bias']}] {
    )
    txt = claude(prompt) or snapshot()
    send_telegram("📊 RECAP DE FIN DE SEMAINE - GOLD\n\n" + txt)


if __name__ == "__main__":
    mode = (sys.argv[1] if len(sys.argv) > 1 else "test").strip()
    if mode == "test":
        send_telegram("✅ Gold Watch est connecté.\n\n" + snapshot())
    elif mode == "alert":
        mode_alert()
    elif mode == "morning":
        mode_morning()
    elif mode == "preview":
        mode_preview()
    elif mode == "weekly":
        mode_weekly()
    else:
        sys.exit(f"Mode inconnu: {mode}")
```